

Нахождение электронной плотности

...

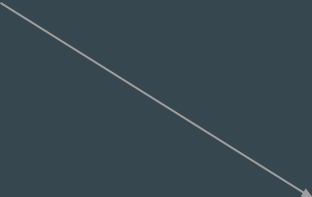
Гуревич Тимофей Б02-312

Теоретическая справка

$$\rho(\mathbf{r}) = N \int |\Psi(\mathbf{r}, \mathbf{r}_2, \dots, \mathbf{r}_N)|^2 d\mathbf{r}_2 \dots d\mathbf{r}_N$$

связь электронной
плотности с волновой
функцией (распределение
вдоль одной оси)

нужно как то найти волновую функцию



метод Хартри-Фока

1.Нахождение с помощью детерминанта
Слейтера

2. Добиваемся минимума энергии

$$E = \frac{\langle \Psi | \hat{H} | \Psi \rangle}{\langle \Psi | \Psi \rangle}$$

$$\Psi(1, 2, \dots, N) = \frac{1}{\sqrt{N!}} \begin{vmatrix} \phi_1(1) & \phi_2(1) & \cdots & \phi_N(1) \\ \phi_1(2) & \phi_2(2) & \cdots & \phi_N(2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_1(N) & \phi_2(N) & \cdots & \phi_N(N) \end{vmatrix}$$

3. Решением является следующее уравнение:

$$\hat{F}\phi_i = \epsilon_i\phi_i$$

Где оператор Фока имеет вид:

$$F = T + V + J - K$$

$$\hat{F} = -\frac{1}{2}\nabla^2 - \sum_A \frac{Z_A}{r_A} + \sum_j (2\hat{J}_j - \hat{K}_j)$$

При решении методом Хартри-
Фока получаем вот:

$$\rho(\mathbf{r}) = 2 \sum_{i=1}^{N/2} |\phi_i(\mathbf{r})|^2$$



$$FC = SC\epsilon$$

$$\phi_i = \sum_{\mu} C_{\mu i} \chi_{\mu}$$

$$P_{\mu\nu} = 2 \sum_i^{\text{occ}} C_{\mu i} C_{\nu i}^*$$

$$E = \sum_{\mu\nu} P_{\mu\nu} (H_{\mu\nu}^{\text{core}} + F_{\mu\nu})$$

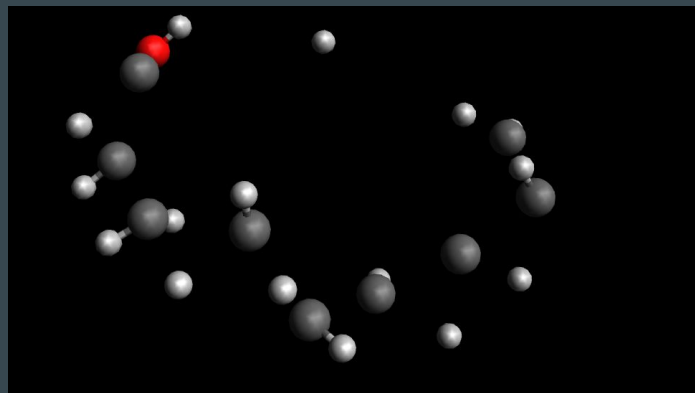
СКОЛЬКО
МОЖНО
ЭТИХ формул



Реализация на питоне

1. По SMILES формуле строим геометрию молекулы (пакет rdkit)

```
def formula_to_3d_geometry(formula):  
    smiles = formula  
  
    mol = Chem.MolFromSmiles(smiles)  
    for bond in mol.GetBonds():  
        print(f"Связь: {bond.GetBeginAtomIdx()}-{bond.GetEndAtomIdx()}, Тип: {bond.GetBondType()}")  
    mol = Chem.AddHs(mol)  
    AllChem.EmbedMolecule(mol)  
    AllChem.MMFFOptimizeMolecule(mol)  
    #tools.cubegen.orbital(mol, 'orbital.cube', mo_coeff, resolution=0.2)  
    display(Draw.MolToImage(mol, size=(400, 400)))  
  
    atoms = []  
    for atom in mol.GetAtoms():  
        pos = mol.GetConformer().GetAtomPosition(atom.GetIdx())  
        atoms.append(f"{atom.GetSymbol()} {pos.x:.6f} {pos.y:.6f} {pos.z:.6f}")  
  
    return "\n".join(atoms)
```



Реализация на питоне

2. Расчет электронной структуры (волновая функция, энергия Фока молекулы)

методом из пакета

собственной реализацией метода
Хартри Фока

```
def calculate_electronic_structure(geometry):  
    #PySCF.  
    mol = gto.M(  
        atom=geometry,  
        basis="6-31g*",  
        spin=0,  
        verbose=4  
    )  
    mf = scf.RHF(mol).run()  
    return mf
```

Своя реализация

а) инициализация

```
class MyRHF:
    def __init__(self, mol):
        self.mol = mol
        self.conv_tol = 1e-8
        self.max_cycle = 50
        self.diis_space = 8
        self.mo_energy = None
        self.mo_coeff = None
        self.e_tot = 0.0
```

б) реализация тела

```
def kernel(self):
    mol = self.mol

    S = mol.intor('int1_ovlp')
    T = mol.intor('int1_kin')
    V = mol.intor('int1_nuc')
    H = T + V

    X = np.linalg.inv(scf.hf.get_ovlp(mol))

    mo_energy, mo_coeff = eigh(H, S)
    nocc = mol.nelectron // 2
    dm = self.make_rdm1(mo_coeff, nocc)

    diis = DIIS()
    diis.space = self.diis_space

    old_dm = dm
    for cycle in range(self.max_cycle):

        J, K = self.get_jk(mol, dm)
        F = H + J - K

        F = diis.update(F, dm, S, mo_coeff)

        mo_energy, mo_coeff = eigh(F, S)
        dm = self.make_rdm1(mo_coeff, nocc)

        delta_dm = np.linalg.norm(dm - old_dm)
        if delta_dm < self.conv_tol:
            logger.note(self, 'SCF converged in %d cycles', cycle+1)
            break
        old_dm = dm

    self.mo_energy = mo_energy
    self.mo_coeff = mo_coeff
    self.e_tot = self.energy_tot(dm, H, F)
    return self.e_tot
```

```

def kernel(self):
    mol = self.mol

    S = mol.intor('int1e_ovlp')
    T = mol.intor('int1e_kin')
    V = mol.intor('int1e_nuc')
    H = T + V

    X = np.linalg.inv(scf.hf.get_ovlp(mol))

    mo_energy, mo_coeff = eigh(H, S)
    nocc = mol.nelectron // 2
    dm = self.make_rdm1(mo_coeff, nocc)

    diis = DIIS()
    diis.space = self.diis_space

    old_dm = dm
    for cycle in range(self.max_cycle):

        J, K = self.get_jk(mol, dm)
        F = H + J - K

        F = diis.update(F, dm, S, mo_coeff)

        mo_energy, mo_coeff = eigh(F, S)
        dm = self.make_rdm1(mo_coeff, nocc)

        delta_dm = np.linalg.norm(dm - old_dm)
        if delta_dm < self.conv_tol:
            logger.note(self, 'SCF converged in %d cycles', cycle+1)
            break
        old_dm = dm

    self.mo_energy = mo_energy
    self.mo_coeff = mo_coeff
    self.e_tot = self.energy_tot(dm, H, F)
    return self.e_tot

```

ПОДРОБНЕЕ ПРО ТЕЛО КЛАССА

$$T_{\mu\nu} = \langle \chi_\mu | -\frac{1}{2} \nabla^2 | \chi_\nu \rangle$$

кин. энергия

$$S_{\mu\nu} = \langle \chi_\mu | \chi_\nu \rangle$$

матрица перекрытия

$$V_{\mu\nu} = \langle \chi_\mu | -\sum_A \frac{Z_A}{|r - R_A|} | \chi_\nu \rangle$$

матрица ядерного
потенциала

Итерационно подбираем матрицу Фока
F, стараясь минимизировать суммарную
энергию

пользуемся diis из ruscф для ускорения
сходимости

```

def make_rdm1(self, mo_coeff, nocc):
    """Создание матрицы плотности"""
    return 2 * np.dot(mo_coeff[:, :nocc], mo_coeff[:, :nocc].T)

def get_jk(self, mol, dm):
    """Вычисление кулоновского и обменного членов"""
    eri = mol.intor('int2e')
    J = np.einsum('pqrs,rs->pq', eri, dm)
    K = np.einsum('prqs,rs->pq', eri, dm)
    return J, K

def energy_tot(self, dm, h1e, fock):
    """Вычисление полной энергии"""
    return np.sum(dm * (h1e + fock)) / 2 + self.mol.energy_nuc()

```

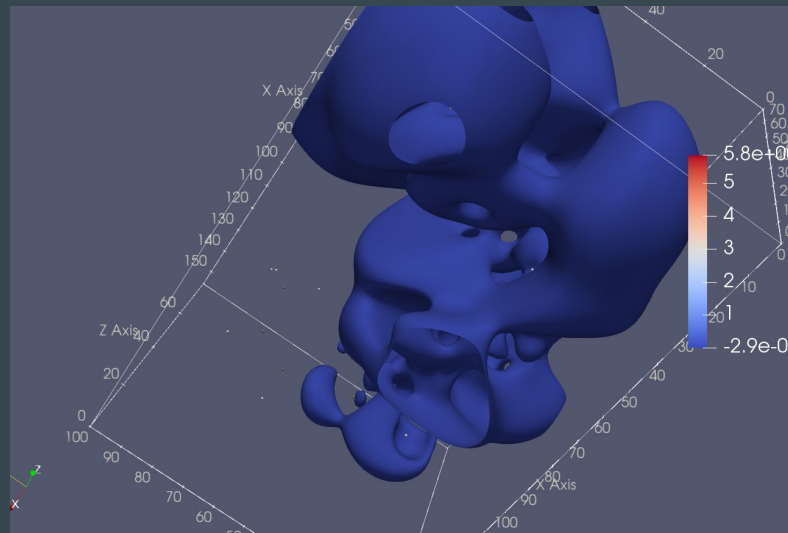
в) вспомогательные функции

```
def calculate_electronic_structure(geometry):
    #СВОя
    mol = gto.M(
        atom=geometry,
        basis="6-31g*",
        spin=0,
        verbose=4
    )
    mf = my.MyRHF(mol)
    mf.kernel()
    return mf
```

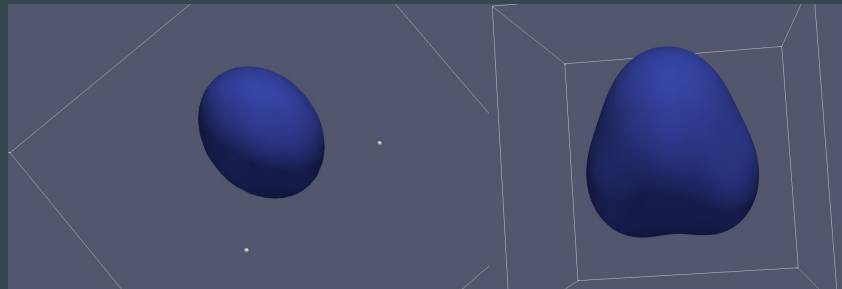
3.С помощью созданного класса считаем структуру и записываем все, что наш метод насчитал

```
for i in range(min(5, mf.mo_coeff.shape[1])):
    cubegen.orbital(
        mf.mol,
        'fixed.cube',
        mf.mo_coeff[:,0],
        nx=60, ny=60, nz=60,
        resolution=0.2,    # Шаг
        margin=5.0        # Отступ
    )
    print(f"{formula}_mo_{i}.cube")
```

4. Открываем с помощью Paraview



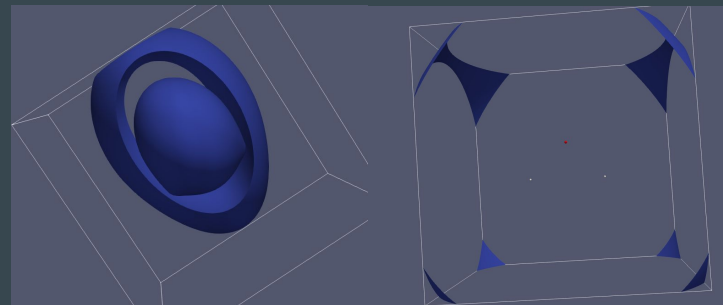
Сравнение для H₂O



1
0
e
-
3

свой

сторонний



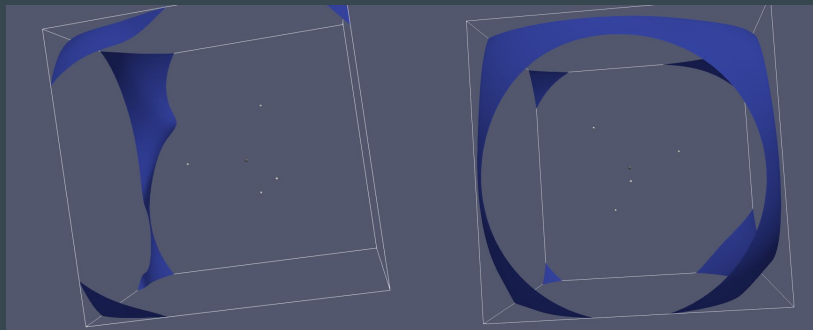
1
0
e
-
7

свой

сторонний



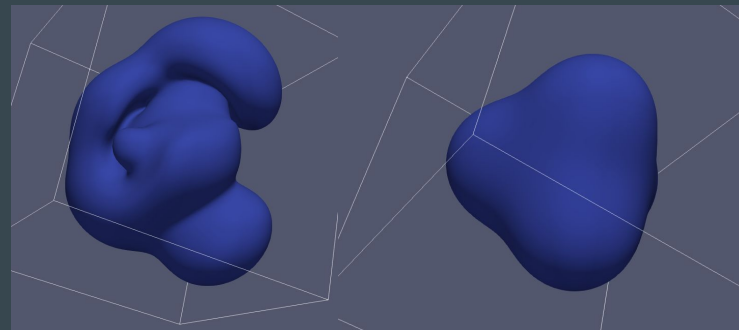
CH₄



1
0
e
-
7

свой

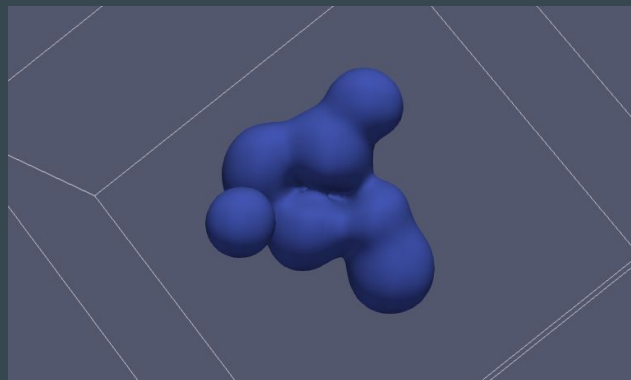
сторонний



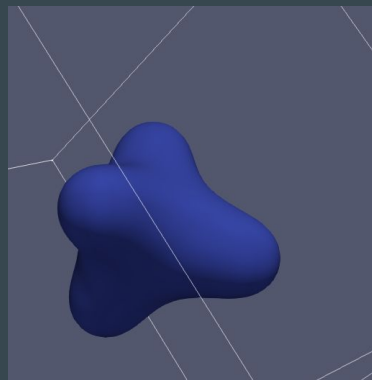
1
0
e
-
3

свой

сторонний



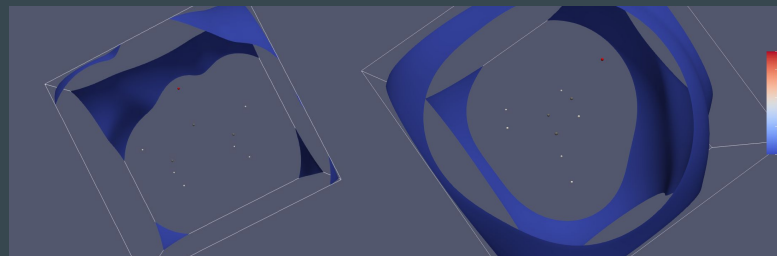
С
В
О
Й



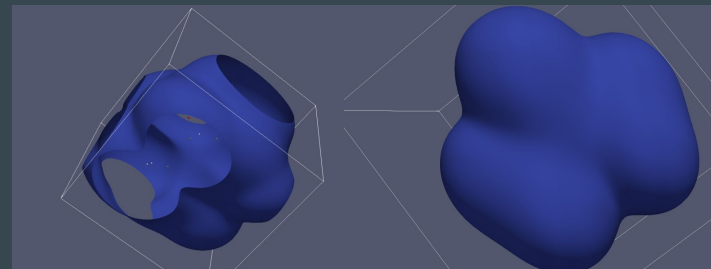
СТОРОННИЙ

10e-2

Ацетон



1
0
e
-
7



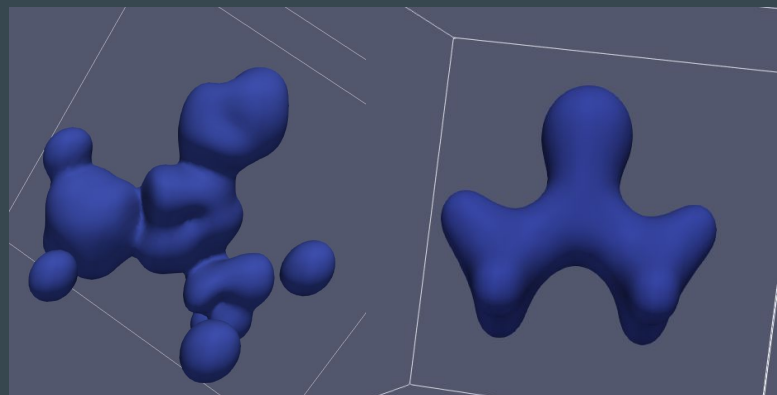
1
0
e
-
4

свой

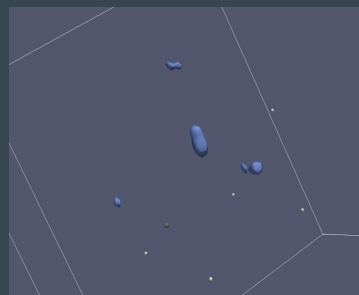
сторонний

свой

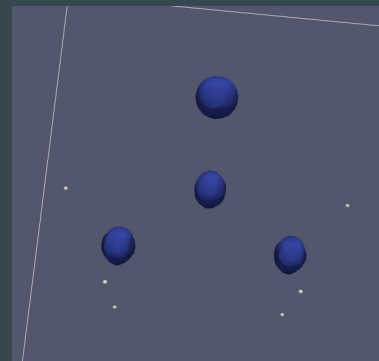
сторонний



1
0
e
-
2



1
0
e
-
1



Замечание: обе реализации работают примерно за одинаковое кол-во времени

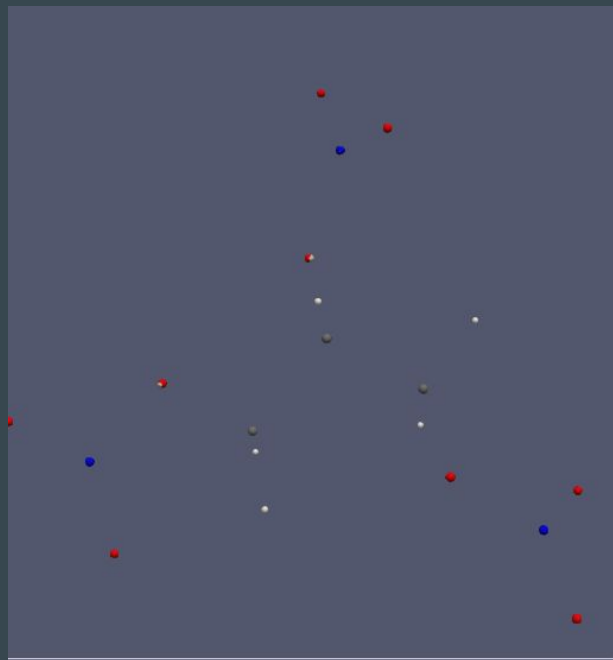
ВЫВОД:



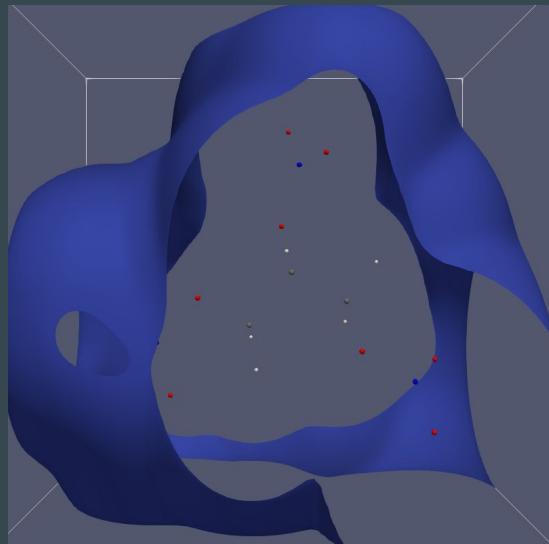
библиотечная реализация метода Хартри-Фока

моя реализация

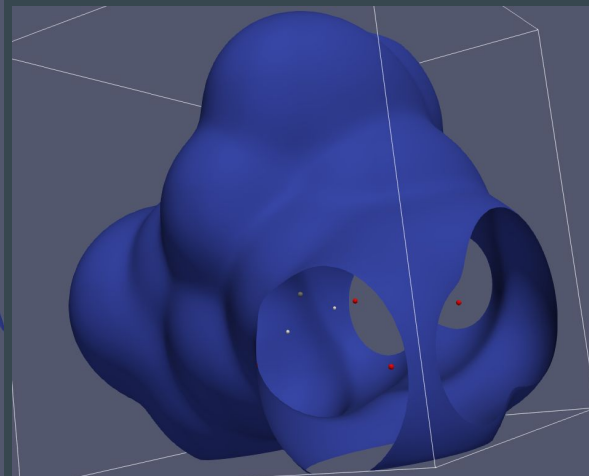
Нитроглицерин



геометрия

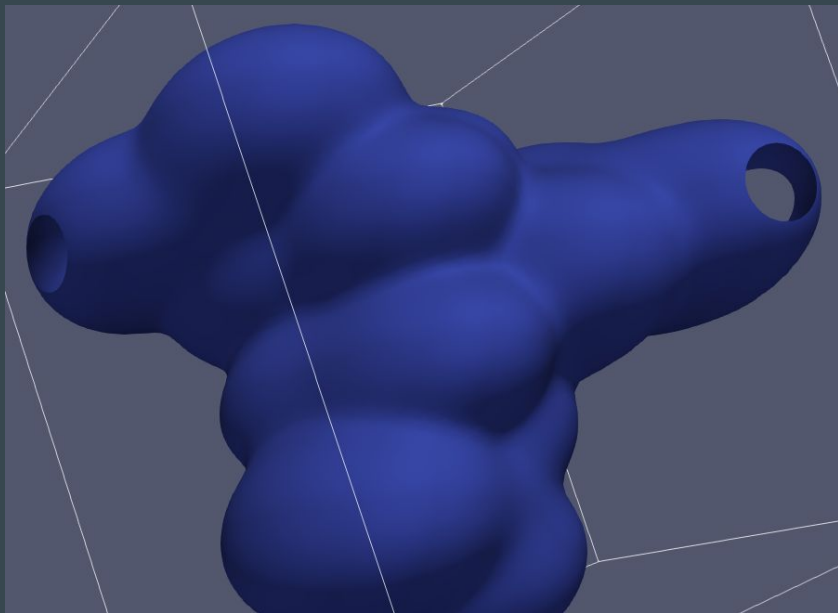


$10e-9$

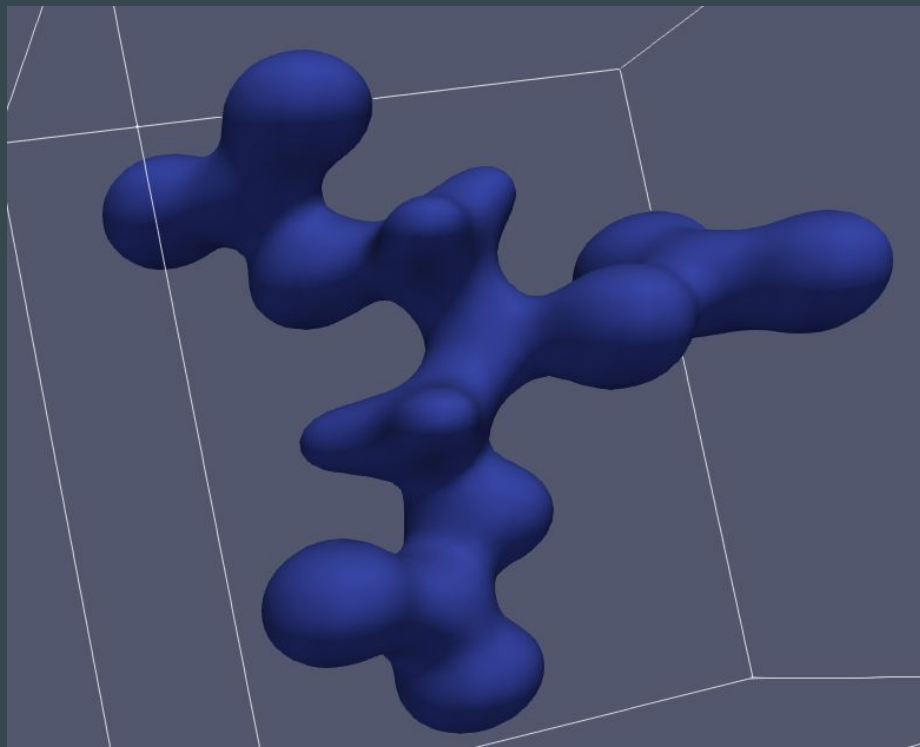


$10e-6$

Нитроглицерин

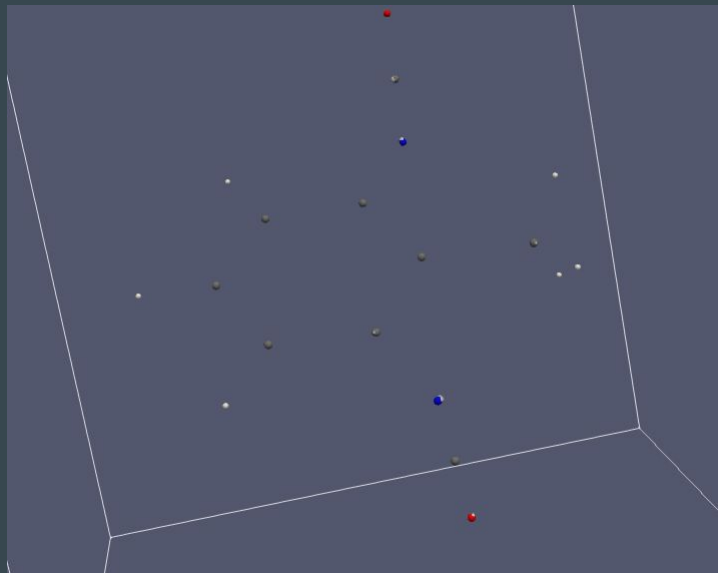


$10e-4$

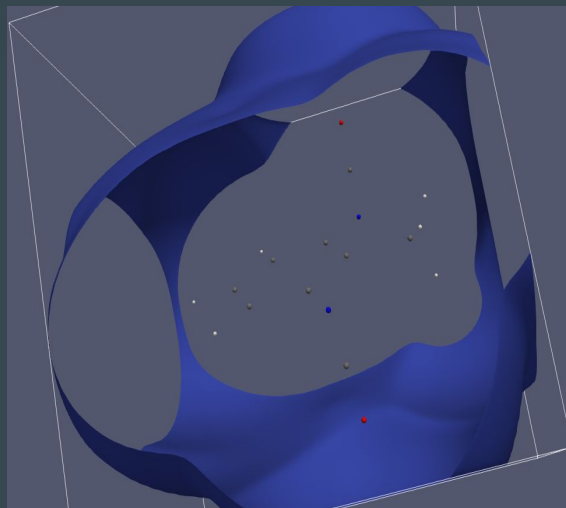


$10e-2$

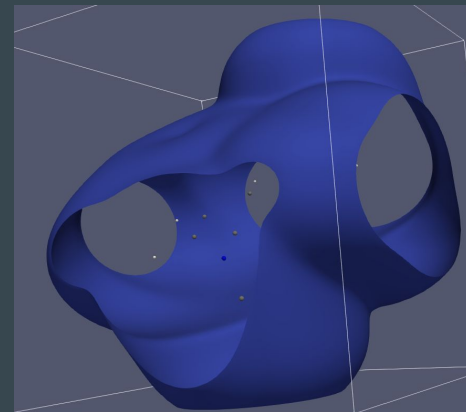
ТОЛУОЛДИИЗОЦИАНАТ



геометрия

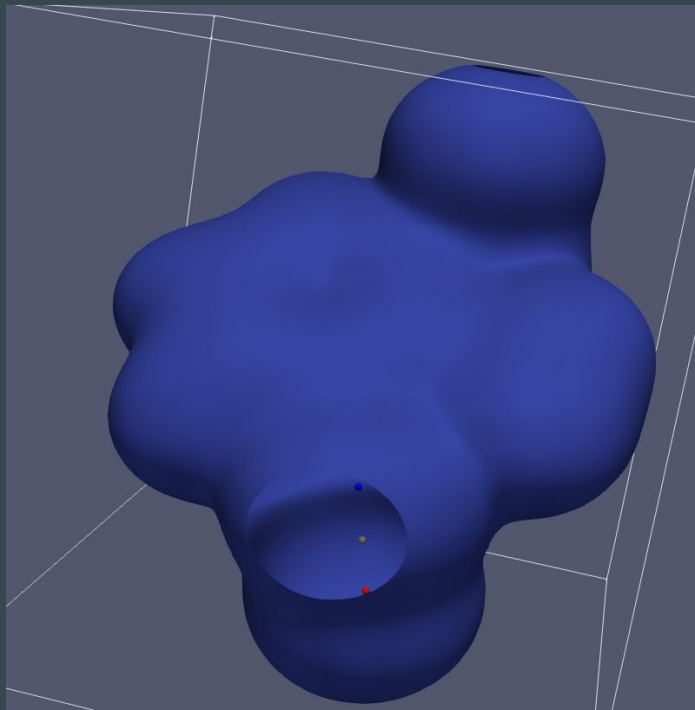


$10e-9$

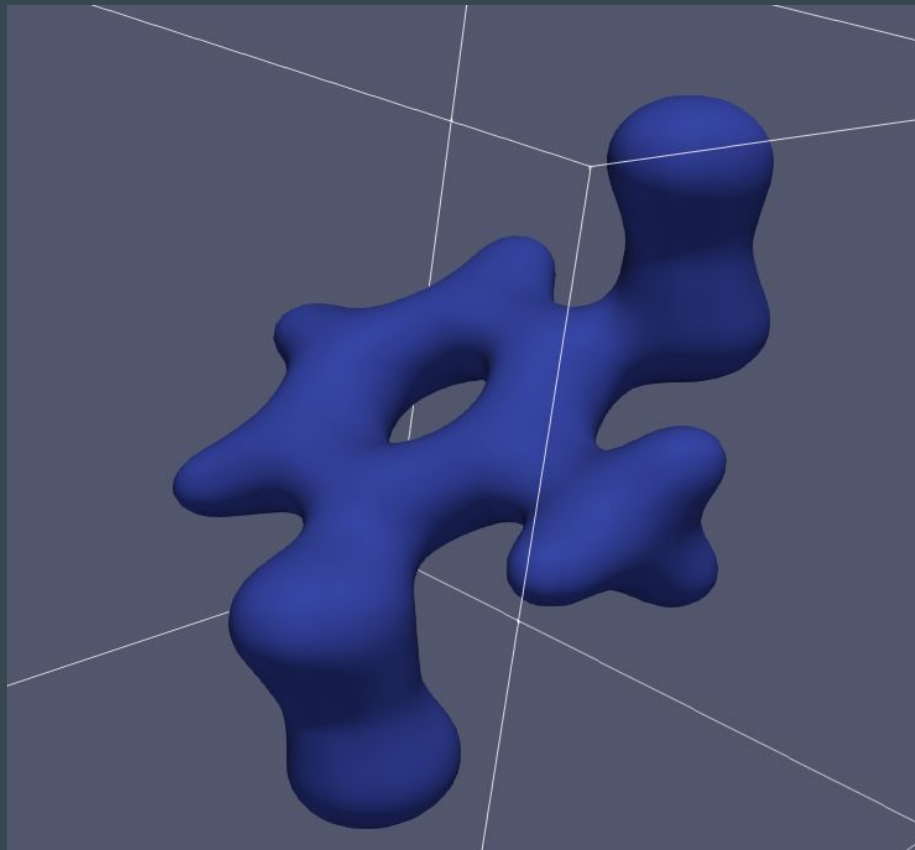


$10e-6$

ТОЛУОЛДИИЗОЦИАНАТ

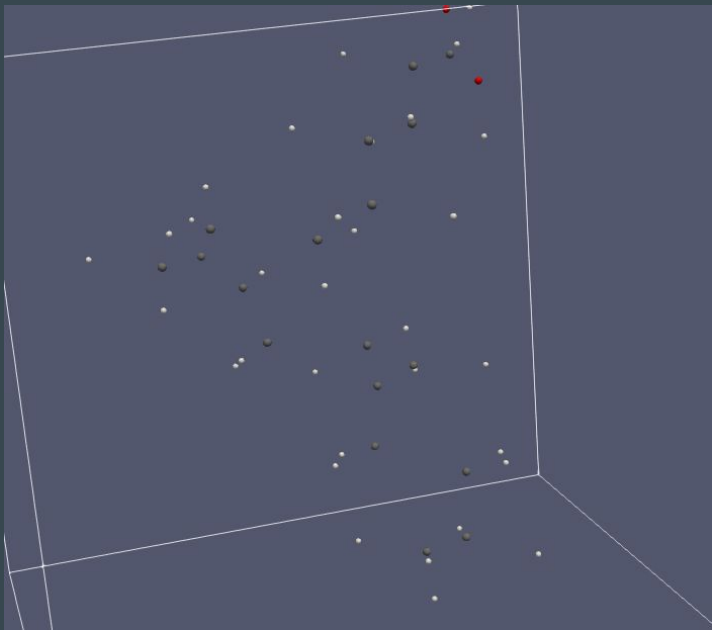


$10e-4$

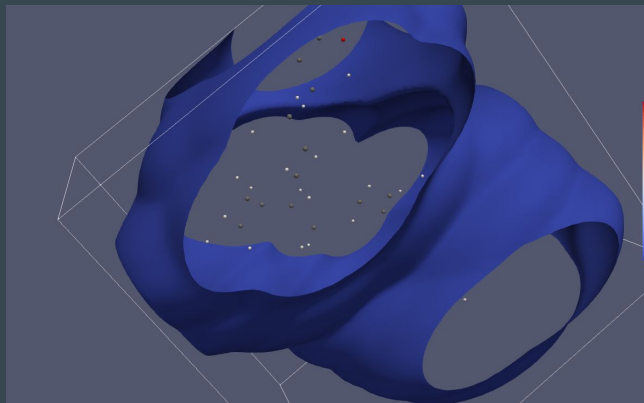


$10e-2$

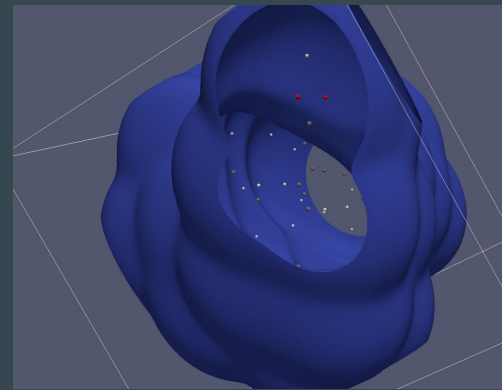
ЛИНОЛЕВАЯ КИСЛОТА



геометрия

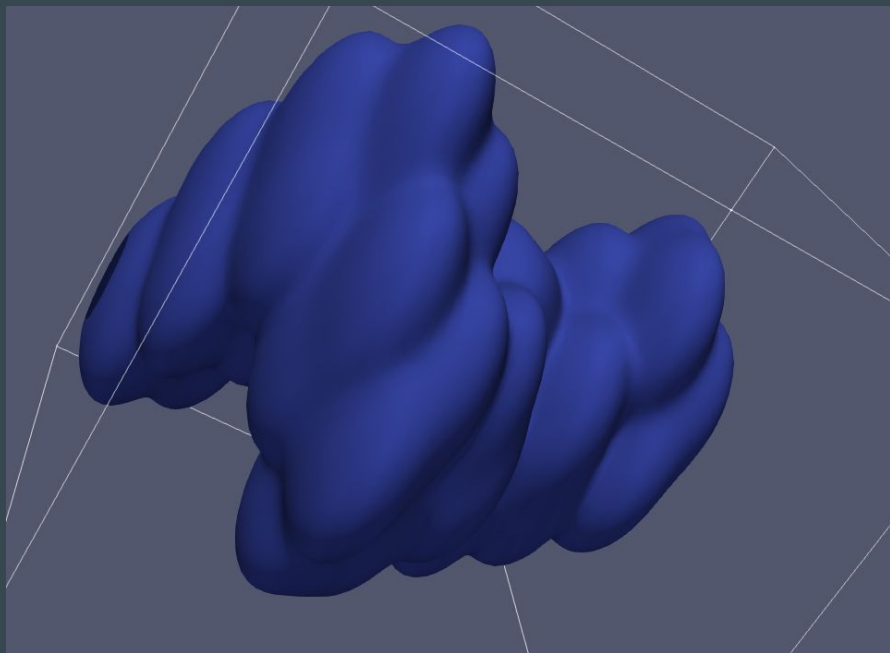


$10e-9$

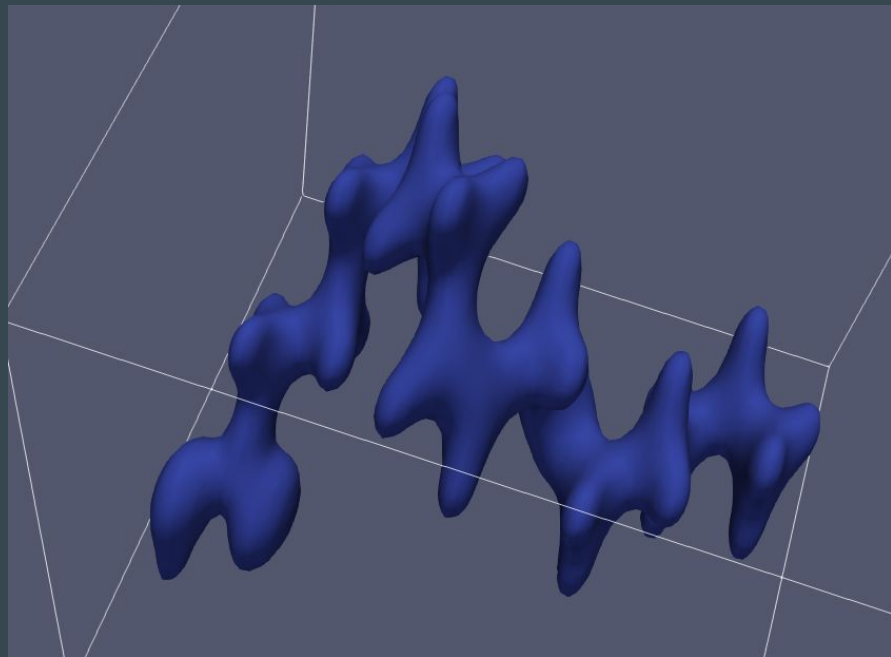


$10e-6$

ЛИНОЛЕВАЯ КИСЛОТА



$10e-4$



$10e-2$